Ferhat Abbas Sétif 1 University
Faculty of Sciences
Department of Computer Science
Université Ferhat Abbas Sétif 1

Practical Work – Lab 3

Convolutional Neural Networks

Prof. Abdelouahab Moussaoui

4th Year Engineering
Option: Artificial Intelligence
Academic Year: 2025-2026

> *"A model is not just built; it is experimented upon, evaluated, and iteratively improved."*
>
> — Pr. Abdelouahab Moussaoui

Preamble

This lab session is designed to complement Chapter 3 (Convolutional Neural Networks). The objective is to move from theory to practice by implementing, training, and evaluating CNNs on real-world datasets. You will start with fundamental exercises to master the TensorFlow/Keras workflow, then explore advanced architectures and techniques, and finally tackle comprehensive mini-projects that simulate real-world challenges.

Lab Objectives

- Implement and train basic CNN models on classic datasets (MNIST, Fashion-MNIST, CIFAR-10).
- Master data augmentation, regularization, and callbacks.
- Experiment with advanced architectures (VGG, ResNet, Inception) and transfer learning.
- Compare model performance, visualize feature maps and filters, and analyze results.
- Complete four mini-projects on diverse image classification tasks.

Tools and Frameworks

- Python 3.x, Google Colab or Jupyter Notebook.
- TensorFlow 2.x / Keras.
- NumPy, Matplotlib, Scikit-learn.
- (Optional) OpenCV, Gradio for deployment.

Submission Guidelines

- Submit a single Jupyter notebook (‘.ipynb‘) for the exercises.
- For the projects, submit a well-documented report (PDF) along with the source code.
- Deadline: To Be Announced (TBA).

Important Notes for Students

- The exercises are designed to progressively build your understanding. You are encouraged to complete as many as possible, but it is not mandatory to finish all of them. Focus on mastering the core concepts; the more you practice, the better you will grasp CNNs.

- Regarding the mini-projects: **two projects are mandatory** (you may choose any two among the four). The remaining two projects can be completed as **bonus work**, which will contribute extra points to your final lab grade. This gives you the opportunity to explore additional topics and improve your score.

Contents

Preamble **2**

1 Part 1: Hands-On Deep Learning (Basic Exercises) **5**
1.1 Exercise 1: Your First CNN on MNIST 5
1.2 Exercise 2: Deep vs. Shallow: Comparing ANN and CNN on Fashion-MNIST 5
1.3 Exercise 3: Effect of Padding and Stride on Real Images 5
1.4 Exercise 4: Data Augmentation to Fight Overfitting 6
1.5 Exercise 5: Visualizing Feature Maps 6
1.6 Exercise 6: Using Callbacks - EarlyStopping and ModelCheckpoint 7
1.7 Exercise 7: Batch Normalization 7
1.8 Exercise 8: Dropout vs No Dropout 7
1.9 Exercise 9: Learning Rate Scheduling 8
1.10 Exercise 10: Model Ensembling 8

2 Part 2: Advanced Deep Learning **8**
2.1 Exercise 11: Transfer Learning with Pretrained VGG16 (Cats vs Dogs) – From Scratch vs Transfer 8
2.2 Exercise 12: Fine-Tuning a Pretrained Model 9
2.3 Exercise 13: Depthwise Separable Convolutions vs Standard Convolutions 9
2.4 Exercise 14: Implementing a Mini Inception Module 10
2.5 Exercise 15: Building a Residual Block from Scratch 10
2.6 Exercise 16: Comparing VGG16, ResNet50, and InceptionV3 10
2.7 Exercise 17: Global Average Pooling vs Flatten 11
2.8 Exercise 18: Visualizing Learned Filters (Kernels) 11
2.9 Exercise 19: Exploring Different Optimizers 11
2.10 Exercise 20: Handling Overfitting with Label Smoothing 12

3 Part 3: Mini-Projects **12**
3.1 Project 1: Multi-Class Food Classification 12
3.2 Project 2: Skin Lesion Classification with Imbalanced Data 12
3.3 Project 3: Natural Scene Classification 13
3.4 Project 4: Flower Species Classification 13

1 Part 1: Hands-On Deep Learning (Basic Exercises)

1.1 Exercise 1: Your First CNN on MNIST

- **Objective:** Build and train a simple CNN to classify handwritten digits.
- **Dataset:** MNIST (available via `tf.keras.datasets.mnist`). Link to dataset info
- **Tasks:**
  1. Load and normalize the data (reshape images to (28,28,1)).
  2. Build a model with two convolutional layers (32 and 64 filters, 3×3 kernel), each followed by max-pooling (2×2).
  3. Add a flatten layer, a dense layer (128 units), and a final softmax layer (10 units).
  4. Compile with 'adam' optimizer and 'sparse_categorical_crossentropy'.
  5. Train for 10 epochs and evaluate on the test set. Report test accuracy.

1.2 Exercise 2: Deep vs. Shallow: Comparing ANN and CNN on Fashion-MNIST

- **Objective:** Compare the performance of a simple Artificial Neural Network (ANN) with a CNN.
- **Dataset:** Fashion-MNIST (`tf.keras.datasets.fashion_mnist`). Dataset link
- **Tasks:**
  1. Build a simple ANN with one hidden layer (256 neurons) and train it.
  2. Build a CNN similar to Exercise 1.
  3. Compare the test accuracy and number of parameters of both models.
  4. Analyze why the CNN performs better for image data.

1.3 Exercise 3: Effect of Padding and Stride on Real Images

- **Objective:** Observe how padding and stride affect the spatial dimensions of feature maps using a real dataset.
- **Dataset:** CIFAR-10 (`tf.keras.datasets.cifar10`).
- **Tasks:**
  1. Load a batch of images from CIFAR-10 (e.g., 8 images) and normalize them.
  2. Define a Conv2D layer with a single 3×3 filter. Experiment with `padding='valid'` and `padding='same'`, and with strides 1 and 2.
  3. Pass the batch through the layer and print the output shape for each combination.

4. Visualize the original images and the resulting feature maps (after convolution) for one combination. What do you observe?

1.4 Exercise 4: Data Augmentation to Fight Overfitting

- **Objective:** Implement data augmentation and observe its impact on validation accuracy.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Build a baseline CNN with 3 convolutional blocks.
  2. Train for 30 epochs and plot training/validation accuracy. Note the overfitting.
  3. Create a new model that includes `RandomFlip()` and `RandomRotation()` layers at the beginning.
  4. Retrain the model with data augmentation. Compare the validation curves.

1.5 Exercise 5: Visualizing Feature Maps

- **Objective:** Visualize the feature maps learned by the first convolutional layer of a CNN.
- **Dataset:** Use the model trained in Exercise 1 (MNIST CNN).
- **Tasks:**
  1. Load the trained model from Exercise 1 (or retrain it if necessary).
  2. Create a new Keras model that outputs the activations of the first convolutional layer. For example:

```
from tensorflow.keras.models import Model
layer_outputs = model.layers[1].output  # first conv layer
feature_map_model = Model(inputs=model.input, outputs=
    layer_outputs)
```

  3. Select one test image (e.g., from the MNIST test set) and preprocess it (normalize, reshape to (1,28,28,1)).
  4. Pass the image through `feature_map_model` to obtain the feature maps (shape: (1, 26, 26, 32) if using valid padding).
  5. Plot all 32 feature maps as a grid of grayscale images using matplotlib. Use `subplot` to arrange them.
  6. Observe and comment: Which features are activated? Do you see edges, corners, or patterns?
  7. Repeat with a different digit and compare the activations.

1.6 Exercise 6: Using Callbacks - EarlyStopping and ModelCheckpoint

- **Objective:** Use Keras callbacks to train more efficiently.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Take the model from Exercise 4 (without augmentation).
  2. Implement `EarlyStopping` (patience=3) to stop training when validation loss stops improving.
  3. Implement `ModelCheckpoint` to save the best model.
  4. Train for 50 epochs and verify that training stops early.

1.7 Exercise 7: Batch Normalization

- **Objective:** Understand the effect of batch normalization on training speed and stability.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Build two identical CNNs: one with batch normalization after each convolutional layer, one without.
  2. Train both for 20 epochs and compare training/validation curves.
  3. Observe the difference in convergence and final accuracy.

1.8 Exercise 8: Dropout vs No Dropout

- **Objective:** Evaluate the regularization effect of dropout.
- **Dataset:** Fashion-MNIST.
- **Tasks:**
  1. Build a CNN with dropout (0.5) after the dense layers, and another without dropout.
  2. Train both and compare overfitting (gap between train and validation accuracy).
  3. Which model generalizes better?

1.9 Exercise 9: Learning Rate Scheduling

- **Objective:** Experiment with learning rate decay to improve training.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Train a CNN with a fixed learning rate (e.g., 0.001).
  2. Train the same model using a learning rate scheduler (e.g., `ExponentialDecay` or `ReduceLROnPlateau`).
  3. Compare loss curves and final accuracy.

1.10 Exercise 10: Model Ensembling

- **Objective:** Improve accuracy by averaging predictions from multiple models.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Train 3 slightly different CNN architectures (e.g., different number of filters).
  2. For a test image, average their softmax outputs and take the argmax.
  3. Compare ensemble accuracy with individual model accuracies.

2 Part 2: Advanced Deep Learning

2.1 Exercise 11: Transfer Learning with Pretrained VGG16 (Cats vs Dogs) – From Scratch vs Transfer

- **Objective:** Compare a CNN built from scratch with a transfer learning approach using VGG16 on the Cats vs Dogs dataset.
- **Dataset:** Cats vs Dogs (subset). Download from Microsoft's Asirra dataset link or use TensorFlow Datasets (`tfds.image.CatsVsDogs`). Use a subset of 2000 images per class for faster training.
- **Tasks:**
  1. **Data preparation:** Load images, resize to 150×150, create training/validation splits (80/20). Normalize pixel values to $[0, 1]$.
  2. **CNN from scratch:** Build a sequential model with:
     - Conv2D(32, 3×3, activation='relu', input_shape=(150,150,3)) + MaxPooling2D(2×2)
     - Conv2D(64, 3×3, activation='relu') + MaxPooling2D(2×2)
     - Conv2D(128, 3×3, activation='relu') + MaxPooling2D(2×2)
     - Flatten + Dense(512, activation='relu') + Dropout(0.5)
     - Dense(1, activation='sigmoid')

     Compile with Adam optimizer and binary crossentropy. Train for 20 epochs and record validation accuracy.
  3. **Transfer learning with VGG16:**
     - Load VGG16 with `include_top=False`, `weights='imagenet'`, `input_shape=(150,150,3)`.
     - Freeze the convolutional base.
     - Add a custom classifier: Flatten (or GlobalAveragePooling2D), Dense(256, activation='relu'), Dropout(0.5), Dense(1, activation='sigmoid').
     - Train for 20 epochs and record validation accuracy.
  4. **Comparison:** Plot training/validation accuracy curves for both models side by side. Compare final test accuracy, number of parameters, and training time.
  5. **Analysis:** Which model performs better? Why does transfer learning help even with a relatively small dataset? Discuss overfitting in the scratch model.

2.2 Exercise 12: Fine-Tuning a Pretrained Model

- **Objective:** Unfreeze some layers of a pretrained model to adapt it better.
- **Dataset:** Cats vs Dogs (same as above).
- **Tasks:**

1. Start from the model in Exercise 11 (the VGG16 version).
2. Unfreeze the last 4 convolutional layers of VGG16.
3. Recompile with a very low learning rate.
4. Continue training and compare final accuracy with the frozen model.

2.3 Exercise 13: Depthwise Separable Convolutions vs Standard Convolutions

- **Objective:** Compare parameter efficiency and performance.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Build a small model using `Conv2D` with 32 filters (3×3). Count parameters.
  2. Build an equivalent model using `SeparableConv2D`.
  3. Compare parameter counts.
  4. Train both on CIFAR-10 for a few epochs and compare accuracy and speed.

2.4 Exercise 14: Implementing a Mini Inception Module

- **Objective:** Build a simplified Inception module using the Keras Functional API.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Create an input layer of shape (32,32,3).
  2. Create three parallel paths: 1×1 conv, 3×3 conv, 5×5 conv (with appropriate padding).
  3. Concatenate the outputs.
  4. Add a few more layers to form a classifier and train it.
  5. Compare its performance with a plain CNN of similar depth.

2.5 Exercise 15: Building a Residual Block from Scratch

- **Objective:** Implement a residual block and build a mini-ResNet.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Create a custom residual block: two convolutional layers with skip connection.
  2. Build a small network with 3 such blocks, followed by global average pooling and a dense layer.
  3. Train and compare with a plain CNN of similar depth.

2.6 Exercise 16: Comparing VGG16, ResNet50, and InceptionV3

- **Objective:** Compare the performance of different pretrained architectures.
- **Dataset:** Cats vs Dogs (or CIFAR-10 resized to 224×224).
- **Tasks:**
  1. Load VGG16, ResNet50, and InceptionV3 with pretrained weights.
  2. Add a custom classifier on top of each.
  3. Train (or fine-tune) them on the dataset.
  4. Compare accuracy, number of parameters, and training time.

2.7 Exercise 17: Global Average Pooling vs Flatten

- **Objective:** Understand the advantage of global average pooling before the final layer.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Build two CNNs identical except one uses `Flatten` and the other uses `GlobalAveragePooling2D` before the dense layer.
  2. Compare the number of parameters and accuracy.
  3. Which one overfits less?

2.8 Exercise 18: Visualizing Learned Filters (Kernels)

- **Objective:** Visualize the filters (kernels) learned by a convolutional layer to understand what patterns they detect.
- **Dataset:** Use the model trained in Exercise 11 (from scratch) or any trained CNN.
- **Tasks:**
  1. Load the trained model.
  2. Extract the weights of the first convolutional layer: `weights = model.layers[0].get_weights()[0]`. This gives a tensor of shape (kernel_height, kernel_width, input_channels, num_filters).
  3. For each filter, normalize the weights to [0,1] and visualize them as small images. If the input has 3 channels (RGB), you can display the filters as color images; if grayscale, display as grayscale.
  4. Plot all filters in a grid using matplotlib. For example, 32 filters in a 6×6 grid.
  5. Observe: Do the filters look like edge detectors, blobs, or random noise? What does this tell you about what the network learns?
  6. (Optional) Repeat for a deeper layer (but note that deeper filters are harder to interpret as they operate on feature maps).

2.9 Exercise 19: Exploring Different Optimizers

- **Objective:** Compare SGD, Adam, and RMSprop on a CNN.
- **Dataset:** Fashion-MNIST.
- **Tasks:**
  1. Build a fixed CNN architecture.
  2. Train it with SGD (momentum=0.9), Adam, and RMSprop.
  3. Compare convergence speed and final accuracy.

2.10 Exercise 20: Handling Overfitting with Label Smoothing

- **Objective:** Apply label smoothing to improve generalization.
- **Dataset:** CIFAR-10.
- **Tasks:**
  1. Train a baseline CNN with standard categorical crossentropy.
  2. Train the same model with label smoothing (using `tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1)`).
  3. Compare validation accuracy and calibration.

3 Part 3: Mini-Projects

3.1 Project 1: Multi-Class Food Classification

- **Objective:** Develop a robust classifier for 10 different food types.
- **Dataset:** Food-101 subset. Available on Kaggle.
- **Problem Statement:** Build a model for a dietary app that recognizes food from user photos with varying quality.
- **Tasks:**
  1. Perform exploratory data analysis (EDA).
  2. Apply heavy data augmentation (rotation, zoom, flip, brightness).
  3. Use transfer learning (EfficientNetB0 or ResNet50) and fine-tune.
  4. Aim for test accuracy $> 85\%$.
  5. Analyze misclassifications (e.g., burger vs hot dog).

3.2 Project 2: Skin Lesion Classification with Imbalanced Data

- **Objective:** Tackle class imbalance in medical imaging.
- **Dataset:** HAM10000. Available on Kaggle.
- **Problem Statement:** Detect melanoma from skin lesions; dataset is highly imbalanced.
- **Tasks:**
  1. Analyze class distribution (bar plot).
  2. Use stratified train/validation/test split.
  3. Start with baseline CNN, observe bias.
  4. Experiment with class weights, augmentation for minority classes, and (optional) GAN-based augmentation.
  5. Evaluate with precision, recall, F1-score, confusion matrix.
  6. Discuss trade-offs for improving recall on melanoma.

3.3 Project 3: Natural Scene Classification

- **Objective:** Classify natural scenes for autonomous drone navigation.
- **Dataset:** Intel Image Classification. Available on Kaggle (6 classes: buildings, forest, glacier, mountain, sea, street).
- **Tasks:**

1. Preprocess (resize to 128×128, normalize).
2. EDA: sample images, class balance.
3. Build a custom CNN from scratch.
4. Apply transfer learning (MobileNetV2) and compare.
5. Use data augmentation.
6. Analyze misclassifications.
7. (Bonus) Visualize feature maps or filters.

3.4 Project 4: Flower Species Classification

- **Objective:** Classify flower species from photographs.
- **Dataset:** Oxford Flowers 102. Available on official site or via TensorFlow Datasets (`tfds.image.OxfordFlowers102`).
- **Problem Statement:** Build a system that can identify flower species for a botanical garden app. The dataset contains 102 species with high intra-class variability.
- **Tasks:**
  1. Load and explore the dataset (display sample images per species).
  2. Handle the large number of classes (102) and potential class imbalance.
  3. Implement a strong baseline using a modern CNN (e.g., ResNet50 or InceptionV3) with transfer learning.
  4. Fine-tune the model and use learning rate scheduling.
  5. Evaluate top-1 and top-5 accuracy.
  6. Use data augmentation to improve generalization (random crops, flips, color jitter).
  7. Analyze which species are most often confused (e.g., different roses) and propose reasons.
  8. (Bonus) Build a simple Gradio app to allow users to upload a flower photo and get a prediction.

> *"The only way to learn a new programming language is by writing programs in it. The only way to learn deep learning is by building models."*